ChatGPT

Repository Tree Map and Slash Notation Feature Audit

Full repository tree

The `chord-sheet-maker-pro` project is structured as follows (directories are in **bold**, files are italicised). Only project files are listed – `node_modules` and build artifacts are omitted for brevity.

- **chord-sheet-maker-pro-main** (root)
- *AGENT_REPORT.md*, *Agent.MD*, *CSMPN-Xproposal.md*, *CSMPUGPROFeature.md*, *Format data OSMD-VexFlow Ecosystem-deep-research-report.md*, *PROJECT_STATUS_AUDIT_2026-03-30.md*, *README.md*: documentation and planning notes.
- *app.html*, *index.html*: static HTML pages for different demos/prototypes.
- **docs**
  - *notagen-integration-opportunities.md*, *vexflow-notation-enhancement-plan.md*: research notes.
- **public**
  - *ug-pro-importer.html*: stand-alone page for importing UltimateGuitar Pro PDFs.
  - **vendor**
  - *README.md*: instructs where to place pdfjs resources for offline use.
- **scripts**
  - *oemer_helper.py*: Python utility for Oemer image processing.
- **src**
  - PDF samples: *Al Green - Love And Happiness - Pro.pdf*, *Alannah Myles - Black Velvet - Pro.pdf*, etc.
  - *main.tsx*: entry point bootstrapping the React app.
  - *App.tsx*: main React component orchestrating import/export, notation rendering, chord-chart modes and UI state.
  - **components** – importer panels for different formats:
  - *UGProImporterPanel.tsx*, *OemerImageImporterPanel.tsx*, *SvgImporterPanel.tsx*.
  - **converters**
  - *musicXMLtochordpro.ts*: converts MusicXML to canonical ChordPro format.
  - **ingest** – preprocessing and import helpers:
  - *abcNormalization.ts*, *batchImportDiagnostics.ts*, *canonicalChart.ts*, *extractTextFromPdf.ts*, *importQuality.ts*, *notagenBridge.ts*, *notagenImportPipeline.ts*, *oemerBridge.ts*, *sniffFormat.ts*, *svgImporter.ts*, *ugProPdfImporter.ts*, *xmlDocumentParser.ts*.
  - **models**
  - *ChordChartModel.ts*: TypeScript interfaces describing the chord chart and tokens.
  - **parsers** – parse different input formats:
  - *abcParser.ts*, *chordProParser.ts*, *csmpnParser.ts*, *gpParser.ts*, *musicXmlToCsmpnFakebook.ts*, *serializeChordPro.ts*.
  - **renderers** – React/SVG renderers:
  - *ChordChart.tsx*: renders chord charts in grid/fake-book or chord-over-lyrics layout.

    - *SlashNotationView.tsx*: **new component** generating slash-notation SVG from a `ChordChartDocument`.
    - **utils**
    - *fretToChord.ts*, *sectionUtils.ts*, *vexflowNotation.js*: helpers for chord inference, section naming and notation drawing.
    - *styles.css*: global CSS, including styles for the slash-notation view and toggle buttons.
- **tests**
    - *validate.html*, *validate.js*, *vexflowNotation.test.mjs*: test harness and a unit test for VexFlow rendering.
    - **fixtures** – placeholder PDFs and expected outputs.
- *tsconfig*.json files: TypeScript configuration for the app, node tools and tests.
- *vite.config.ts*: Vite build configuration.
- *xmlsamples.zip*: collection of sample MusicXML files.

CI/CD inspection: There is **no** `.github/workflows` **directory** in the repository. This means the project currently lacks GitHub Actions or any other automated CI/CD pipeline. Without a CI workflow, changes are not automatically linted, tested or built when pushed. Introducing a workflow (e.g., running `npm run test` or `npm run build` on pull requests and pushes) would help catch issues like missing side panels or integration errors early.

Slash Notation feature integration

The new slash-notation feature is implemented through a combination of a new renderer component, UI state additions and styling. Key changes are summarised below.

`src/renderers/SlashNotationView.tsx` (new 310-line file)

SlashNotationView is a standalone React component that takes a `ChordChartDocument` and renders it as Real Book–style slash rhythm notation. Important implementation details:

- **Data pipeline:** The component maps the chord chart into `SlashSection` objects containing `SlashMeasure` objects. Helpers distribute chords evenly across beats according to the time signature, handle barline tokens (`|`, `||`, `|:` and `:|`) and respect repeat starts/ends. It supports both grid/fake-book lines and chord-over-lyrics lines by grouping chords into measures when barlines are absent.
- **SVG rendering:** It uses pure SVG – there is no dependency on VexFlow or canvas. Functions draw staff lines, custom parallelogram slash noteheads, barlines (including single, double and repeat markers), chord symbols positioned above beats and section labels above each system. Layout constants define staff spacing (five horizontal lines with 8 px gaps), chord area height, system spacing and page width.
- **Responsiveness:** The SVG is wrapped in a viewbox and the component accepts a `measuresPerRow` prop (default 4, configurable between 1–8) to control how many measures appear per row. It also honours transpose steps via the existing `transposeChord()` helper.

This file is central to the new feature and matches the Real Book slash-notation examples provided by the user.

`src/App.tsx` modifications

`App.tsx` coordinates the application's modes (notation, chord chart, UG-Pro importer, OEMER image importer, SVG importer). The slash-notation feature touches several areas:

1. **Imports and state:** The file imports `SlashNotationView` and declares two new pieces of state:

```
const [showSlashNotation, setShowSlashNotation] = useState(false);
const [slashMeasuresPerRow, setSlashMeasuresPerRow] = useState(4);
```

2. **Top-bar controls:** When in chord-chart mode, the top bar includes a "Slash Notation" toggle button. When active, an inline input appears allowing users to pick 1–8 measures per row. The button text toggles between "Slash Notation" and "Back to Chord Chart".
3. **Content viewport:** The main `<section>` that normally renders `<ChordChart>` now conditionally renders `<SlashNotationView>` when `showSlashNotation` is `true`.
4. **Side panel:** A new section titled *"Slash Notation"* appears in the right-hand side panel when in chord-chart mode. It contains a large "𝄞 Convert to Slash Notation" button, again toggling `showSlashNotation`. When active, a number input appears ("Measures per row").
5. **Reset:** The `clearAll()` helper resets the new state variables (`showSlashNotation` and `slashMeasuresPerRow`).

These changes ensure that the slash-notation mode is accessible from both the top toolbar and the side panel.

`src/styles.css` additions

New classes style the slash-notation view and toggle buttons:

- `.sn-view`, `.sn-header`, `.sn-title`, `.sn-artist`, `.sn-meta`, `.sn-svg`: styling for the slash-notation page (white background, serif fonts, header formatting and responsive SVG sizing).
- `.btn-slash-notation` and `.btn-slash-notation.active`: purple-tinted buttons used in the side panel; `.btn-slash-notation--top` adapts sizes for the top bar.
- `.sn-mpr-row` and `.top-slash-mpr`: layout for the "measures per row" input.

Why the side panel might not appear

The side panel is always rendered when the app is in chord-chart mode (`<aside className="side-panel">`). The slash-notation section inside it is shown only when `chartDocument` is non-null (i.e., after a chord chart is loaded). If you cannot see the side panel:

- Ensure you have imported a chord chart (e.g., load a `.pro`, `.txt`, `.xml` or `.pdf` file). The side panel is hidden in notation mode and empty mode.
- The side panel might be off-screen on small screens; it uses CSS grid to occupy a column on the right. Check for horizontal scrolling or adjust the window width.
- If the app hasn't been rebuilt, the old bundle may not include these changes. Running `npm install` and `npm run dev` (or the appropriate build command) should pick up the new code.

CI/CD bottlenecks

The repository lacks any continuous-integration workflows (there is no `.github/workflows` directory). As a result, there is no automated pipeline to:

- Run TypeScript compile checks or lint the code.
- Execute the existing unit test (`tests/vexflowNotation.test.mjs`) or any new tests for the slash-notation feature.
- Build and deploy the app automatically (e.g., to GitHub Pages or a static host).

To improve reliability and catch integration issues early, consider creating a workflow such as:

```
name: CI
on: [push, pull_request]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: 18
      - run: npm ci
      - run: npm run test
      - run: npm run build
```

This would install dependencies, run the test suite and produce a production build on each push/PR.

Conclusion

The slash-notation feature has been integrated into the React application via the new `SlashNotationView.tsx` component, additional UI state in `App.tsx`, and corresponding CSS. The feature appears properly wired into both the top bar and side panel. However, if you cannot find the side-panel controls, ensure you are in chord-chart mode and have re-built the application. Additionally, the repository currently lacks a CI/CD workflow, which could have prevented incomplete or inconsistent integrations. Adding a GitHub Actions workflow would improve the development process and help catch issues before deployment.
